

# LAB RECORD

Computer Science & Engineering  
KMEA Engineering College | KTU | S2 CSE

---

Codes • Expected Output • Questions & Answers • Viva Questions

| Exp No. | Experiment Title                         |
|---------|------------------------------------------|
| 1       | Familiarisation of Computer Fundamentals |
| 2       | Familiarisation of Boot Process          |
| 4       | Shell Programming in Linux               |
| 5       | Familiarisation of Basic Linux Commands  |

# EXPERIMENT 1: FAMILIARISATION OF COMPUTER FUNDAMENTALS

**Problem Definition:** To familiarize computer hardware components.

## ■ Theory Summary

| Component            | Description                                                                                                                                                                  |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Computer             | Electronic device that accepts data as input, processes it, and gives desired output. Main characteristics: Speed, Accuracy, Diligence, Storage Capability, and Versatility. |
| CPU Box              | Computer case/chassis — the enclosure containing most of the hardware of a personal computer.                                                                                |
| Motherboard          | Main circuit board supporting communication between different electrical components; houses CPU, memory, and other components.                                               |
| CPU                  | Central Processing Unit — processes data, controls/coordinates/supervises computer operations. Contains CU (Control Unit) and ALU (Arithmetic Logic Unit).                   |
| NIC (Interface Card) | Network Interface Card — hardware component present on the computer; connects different networking devices.                                                                  |
| Card Slot            | A special slot for inserting a memory card; allows expansion of built-in memory.                                                                                             |
| Hard Disk            | Non-volatile storage device that stores data on a computer/electronic device; retains data even when power is removed.                                                       |
| Cables               | Conductors covered in plastic; power cable (supplies power) and data cable (creates communication between devices).                                                          |
| SMPS                 | Switched Mode Power Supply — power electronics topology with two stages: AC→DC, then DC→desired DC output voltage.                                                           |
| Various Ports        | Connections between external and internal I/O devices (keyboards, motherboards) using cables. Can be virtual or physical.                                                    |
| Input Devices        | Equipment used to provide data and control signals; e.g., keyboard, mouse, scanner, camera, joystick, microphone.                                                            |
| Output Devices       | Hardware that converts data into human-perceptible form; e.g., printer, monitor.                                                                                             |
| Storage Devices      | 5 types: Primary, Magnetic, Flash Memory, Optical, Cloud & Virtual Storage.                                                                                                  |
| Bus                  | High-speed internal connection transferring data/signals/power between components; made of hardware (wires) and software (protocols).                                        |
| Firmware             | Software embedded in a device's hardware to control how it functions.                                                                                                        |

## ■ Expected Questions & Answers

### Q: What is a computer?

A: A computer is an electronic device that accepts data as input, performs processing on the data, and gives the desired output. It is also called an electronic data processing machine.

### Q: What are the main characteristics of a computer?

A: Speed, Accuracy, Diligence, Storage Capability, and Versatility.

### Q: What is a CPU?

A: CPU (Central Processing Unit) processes data and controls/coordinates the operations of a computer. It consists of two important units: CU (Control Unit) and ALU (Arithmetic Logic Unit).

### Q: What is a Motherboard?

A: A motherboard is a circuit board inside computing systems (including personal computers, smart TVs, smart monitors) that supports communication between different electrical components and houses components such as CPU, memory, etc.

### Q: What is SMPS?

A: Switched Mode Power Supply is a power electronics topology with two power stages. The first stage converts AC supply into DC and the second stage converts DC voltage to the desired DC output voltage.

**Q: What is a NIC?**

A: A Network Interface Card is a hardware component present on a computer. It connects different networking devices such as computers and servers to share data, and supports I/O interrupt, Direct Access (DMA) interface, partitioning, and data transmission.

**Q: What is a Hard Disk?**

A: A hard disk (also known as hard disk drive) is a device that stores data on a computer or other electronic device. It's a non-volatile storage device, meaning it can retain data even when the device is powered off.

**Q: What are the types of storage devices?**

A: 1. Primary Storage Devices, 2. Magnetic Storage Devices, 3. Flash Memory Devices, 4. Optical Storage Devices, 5. Cloud and Virtual Storage.

**Q: What is a Bus in a computer?**

A: A bus is a high-speed internal connection that transfers data, signals, or power between components in a computer or between computers. Buses are made up of hardware components like wires and optical fibers as well as software like communication protocols.

**Q: What is firmware?**

A: Firmware is a type of software that is embedded in a device's hardware to control how it functions.

**Q: What is the difference between input and output devices?**

A: Input devices provide data and control signals to an information processing system (e.g., keyboard, mouse, scanner). Output devices convert data into a human-perceptible or machine-readable form (e.g., printer, monitor).

## ■ Viva Questions

**★ What does ALU stand for and what is its function?**

→ ALU stands for Arithmetic Logic Unit. It performs arithmetic operations (addition, subtraction, multiplication, division) and logical operations (AND, OR, NOT comparisons) inside the CPU.

**★ What is the difference between RAM and ROM?**

→ RAM (Random Access Memory) is volatile — it loses data when power is off. ROM (Read Only Memory) is non-volatile — it retains data permanently and typically stores firmware/BIOS.

**★ What is the difference between primary and secondary storage?**

→ Primary storage (RAM) is directly accessible by the CPU and is volatile. Secondary storage (hard disk, SSD) is non-volatile, slower, and used for long-term data storage.

**★ Name any three ports found on a computer.**

→ USB port (data transfer), HDMI port (audio/video output), Ethernet port (network connection), Audio jack, VGA port.

**★ What is the full form of SMPS?**

→ Switched Mode Power Supply.

**★ What does a card slot do?**

→ A card slot is a special slot for inserting a memory card, allowing you to expand the built-in memory of a device.

**Conclusion:** Familiarized with various hardware components of a desktop personal computer.

# EXPERIMENT 2: FAMILIARISATION OF BOOT PROCESS

**Problem Definition:** To understand the sequence of operations during the boot process and explore components involved in starting a computer.

## Materials Required

- A computer with access to the BIOS/UEFI
- Bootable device (e.g., Linux or Windows installation media)
- Access to a virtual machine environment (e.g., VirtualBox or VMware)
- A monitor, keyboard, and mouse

## Theory

The boot process is the sequence of events that occurs from powering on a computer to loading the Operating System (OS). It typically involves the following steps:

| Step | Name            | Description                                                               |
|------|-----------------|---------------------------------------------------------------------------|
| 1    | POST            | Power-On Self-Test — Hardware checks performed by the BIOS/UEFI           |
| 2    | BIOS/UEFI Setup | Loading BIOS/UEFI Setup — Note the key displayed during the splash screen |
| 3    | Boot Order      | BIOS/UEFI checks boot priority (HDD, USB, CD-ROM) to find bootable device |
| 4    | Bootloader      | MBR/GPT loads bootloader (e.g., GRUB) from the primary boot device        |
| 5    | OS Loading      | Bootloader loads the OS kernel into memory                                |
| 6    | Kernel Init     | Kernel initializes hardware components and loads necessary drivers        |
| 7    | OS Ready        | Operating System starts and user login screen appears                     |

## Procedure Parts

- Part 1 – BIOS/UEFI Exploration:** Note the key displayed during splash screen. Explore BIOS/UEFI settings: identify boot order, hardware diagnostics, and system time. Change boot priority to USB device temporarily (if allowed). Record settings related to the boot process such as secure boot and boot device options.
- Part 2 – Observing the Bootloader:** Insert a bootable USB (e.g., Ubuntu live media). Restart the system and observe how the bootloader (e.g., GRUB) appears. Navigate through bootloader options such as safe mode or recovery mode. Note the configuration of the bootloader menu.
- Part 3 – Virtual Machine Experiment:** Create a virtual machine and assign a bootable ISO image as the primary boot device. Start the virtual machine and observe the boot sequence in a controlled environment. Use commands like `dmesg` or `journalctl` (in Linux) to view boot logs and identify steps in the process.
- Part 4 – Understanding Kernel Initialization:** Boot into recovery mode using the bootloader. Observe how the kernel initializes hardware components and loads necessary drivers. Access the command line and review boot logs using commands:

```
css
copy code
dmesg | less
journalctl -b
```

## Expected Questions & Answers

**Q: What is the boot process?**

A: The boot process is the sequence of events that occurs from powering on a computer to loading the Operating System (OS). It involves POST, BIOS/UEFI loading, bootloader execution, kernel initialization, and OS startup.

**Q: What is POST?**

A: POST (Power-On Self-Test) is hardware checks performed by the BIOS/UEFI when the computer is first powered on to ensure all hardware components are functioning correctly.

**Q: What is BIOS?**

A: BIOS (Basic Input/Output System) is firmware stored on a chip on the motherboard. It initializes and tests hardware during the boot process and provides runtime services for the OS.

**Q: What is UEFI and how is it different from BIOS?**

A: UEFI (Unified Extensible Firmware Interface) is a modern replacement for BIOS. It supports larger hard drives (>2TB), faster boot times, a graphical interface, Secure Boot, and network capabilities during boot.

**Q: What is a Bootloader?**

A: A bootloader is a small program that loads the operating system into memory. Examples include GRUB (for Linux) and Windows Boot Manager. It is stored in the MBR or the EFI partition.

**Q: What is GRUB?**

A: GRUB (Grand Unified Bootloader) is a bootloader used primarily in Linux systems. It allows users to choose between multiple operating systems or kernel versions during boot.

**Q: What is the difference between MBR and GPT?**

A: MBR (Master Boot Record) supports disks up to 2TB and up to 4 primary partitions. GPT (GUID Partition Table) supports disks larger than 2TB and up to 128 partitions, and is used with UEFI.

**Q: What command is used to view boot logs in Linux?**

A: 'dmesg' displays kernel ring buffer messages from boot. 'journalctl -b' shows the current boot log. 'journalctl -b -1' shows the previous boot log.

**Q: What is Secure Boot?**

A: Secure Boot is a UEFI feature that prevents unauthorized operating systems and software from loading during the boot process by verifying digital signatures.

**Q: What is a virtual machine?**

A: A virtual machine (VM) is a software emulation of a physical computer that runs an OS and applications as if it were a real machine. Examples: VirtualBox, VMware.

## ■ Viva Questions

★ **What happens if POST fails?**

→ If POST fails, the computer either beeps (beep codes) or displays an error message on screen, and the boot process halts. The system cannot proceed to load the OS.

★ **What key is commonly used to enter BIOS/UEFI setup?**

→ Common keys include Del, F2, F10, F12, or Esc depending on the manufacturer. The key is usually shown briefly during the splash/POST screen.

★ **What is the boot order?**

→ Boot order is the sequence in which the BIOS/UEFI checks devices for a bootable operating system. Typically: HDD → USB → CD/DVD. Can be changed in BIOS settings.

★ **What is the role of the kernel in the boot process?**

→ The kernel is the core of the OS. During boot, it initializes hardware components, loads device drivers, mounts the root filesystem, and starts system processes (init/systemd).

★ **What is 'dmesg' used for?**

→ dmesg (diagnostic messages) displays messages from the kernel ring buffer, useful for diagnosing hardware issues and viewing the boot sequence messages.

**Conclusion:** Summarized the key learnings about the boot process, emphasizing the role of hardware, firmware, and software components in ensuring a successful system startup.

## EXPERIMENT 4: SHELL PROGRAMMING IN LINUX

**Problem Definition:** Write and execute various shell scripts in Linux to practice basic to intermediate shell programming concepts.

### ■ Introduction

Shell programming (shell scripting) involves writing scripts using shell commands. The most common shell in Linux is Bash (Bourne Again SHell). Every script starts with a shebang line: **#!/bin/bash**. To make a script executable: **chmod +x filename.sh** and run with **./filename.sh**

#### Program 1: Program to Display "Hello World"

**Code:**

```
#!/bin/bash
echo "hello world"
```

**Run:** ./new.sh

**Expected Output:**

```
hello world
```

**Questions & Answers:**

**Q: What does 'echo' do in shell?**

A: The 'echo' command prints text or variables to the standard output (terminal/screen).

**Q: What is '#!/bin/bash' called?**

A: It is called a shebang (or hashbang). It tells the OS which interpreter to use to run the script.

**Q: How do you make a shell script executable?**

A: Use the command: `chmod +x filename.sh`

**Q: How do you run a shell script?**

A: Using `./filename.sh` after giving execute permission, or using `bash filename.sh`

#### Program 2: Program to Store a Variable and Display It

**Code:**

```
#!/bin/bash
J=Superman
echo "Batman vs $J"
```

**Run:** ./script.sh

**Expected Output:**

```
Batman vs Superman
```

**Questions & Answers:**

**Q: How do you assign a variable in shell?**

A: Using `variable_name=value` (no spaces around '='). Example: `name=John`

**Q: How do you access a variable's value in shell?**

A: By prefixing the variable name with '\$'. Example: `echo $name` or `echo ${name}`

**Q: Are there spaces allowed around '=' when assigning variables?**

A: No! In shell, `name = value` (with spaces) will cause an error. It must be `name=value`.

#### Program 3: Program to Read Name from User and Display

**Code:**

```
#!/bin/bash
read name
```

```
echo "u hello & hello"
```

**Run:** ./script.sh

**Expected Output:**

```
hello hello
```

**Questions & Answers:**

**Q: What does the 'read' command do?**

A: The 'read' command takes input from the user (stdin) and stores it in a variable. Example: read name

**Q: How do you display a prompt with read?**

A: Use 'read -p "Enter name: " name' to show a prompt before reading input.

---

#### Program 4: Program to Check Whether a String Matches

**Code:**

```
#!/bin/bash
echo "enter your name"
read name
if [ $name = "linux" ]
then echo "matches"
else echo "no match"
fi
```

**Run:** ./script.sh

**Expected Output:**

```
enter your name
asdalghljghi
no match
```

**Questions & Answers:**

**Q: How do you compare strings in shell?**

A: Using [ \$var = "string" ] or [ \$var == "string" ] inside an if condition.

**Q: What is the syntax of an if-else in shell?**

A: if [ condition ]; then ... else ... fi

**Q: Why do we need spaces inside [ ] in shell?**

A: The [ ] is actually a command in shell. Spaces are required around the condition and brackets for proper parsing.

---

#### Program 5: Program to Check Voting Eligibility

**Code:**

```
#!/bin/bash
echo "enter your age"
read age
if [ $age -ge 18 ]
then echo "you can vote"
else echo "you cannot vote"
fi
```

**Run:** ./script.sh

**Expected Output:**

```
enter your age
20
you can vote
```

**Questions & Answers:**



**Q: What does '-ge' mean in shell?**

A: '-ge' means 'greater than or equal to'. It is used for numeric comparisons in shell.

**Q: What are the numeric comparison operators in shell?**

A: -eq (equal), -ne (not equal), -lt (less than), -le (less than or equal), -gt (greater than), -ge (greater than or equal)

**Q: How is numeric comparison different from string comparison in shell?**

A: For numbers, use -eq, -ne, -lt, -gt etc. For strings, use =, !=, <, > inside [[ ]] or [ ].

---

**Program 6: Program to Find Sum of Two Numbers**

**Code:**

```
#!/bin/bash
echo "enter a number"
read a
echo "enter 2nd number"
read b
Sum=$((a + b))
echo "Sum = $((a + b))"
```

**Run:** ./script.sh

**Expected Output:**

```
enter a number
5
enter 2nd number
3
Sum = 8
```

**Questions & Answers:****Q: How do you perform arithmetic in shell?**

A: Using `$( ( expression ) )`. Example: `result=$((a + b))`

**Q: Can you use 'expr' for arithmetic?**

A: Yes. Example: `result=$(expr $a + $b)`. But `$( ( ) )` is preferred in modern bash.

**Q: What is the difference between \$var and \${var}?**

A: `$var` accesses a variable's value (as string), while `${var}` treats the content as arithmetic.

---

**Program 7: Program to Check Odd or Even**

**Code:**

```
#!/bin/bash
echo "enter the number"
read number
if [ $((number%2)) -eq 0 ]
then echo "the number is even"
else echo "the number is odd"
fi
```

**Run:** ./script.sh

**Expected Output:**

```
enter the number
6
the number is even
```

**Questions & Answers:****Q: What is the modulo operator in shell?**

A: % is the modulo operator. It gives the remainder of division. Example: `7%2 = 1`

**Q: How do you check if a number is divisible by another?**

A: Check if the modulo equals zero: [  $((n \% d)) -eq 0$  ]

### Program 8: Program to Print Numbers Using 'for' Loop

Code:

```
#!/bin/bash
echo "enter the limit"
read limit
for i in $(seq 1 $limit)
do
echo $i
done
```

Run: ./script.sh

Expected Output:

```
enter the limit
2
1
2
```

Questions & Answers:

**Q: What is the syntax of a 'for' loop in shell?**

A: for variable in list; do ... done. Example: for i in 1 2 3; do echo \$i; done

**Q: What does 'seq' do?**

A: The 'seq' command generates a sequence of numbers. Example: seq 1 5 generates 1 2 3 4 5

**Q: What is another way to write a for loop in bash?**

A: for ((i=1; i<=limit; i++)); do echo \$i; done — C-style for loop syntax.

### Program 8b: Program to Print Numbers Using 'while' Loop

Code:

```
#!/bin/bash
echo "enter the limit"
read n
i=4
while [ $i -le $n ]
do
echo $i
i=$((i+1))
done
```

Run: ./script.sh

Expected Output:

```
enter the limit
6
4
5
6
```

Questions & Answers:

**Q: What is the syntax of a 'while' loop in shell?**

A: while [ condition ]; do ... done

**Q: How do you increment a variable in shell?**

A: Using i=\$((i+1)) or ((i++)) in bash.

**Q: What is the difference between 'for' and 'while' loops?**

A: 'for' loop iterates over a known list/range. 'while' loop continues as long as a condition is true — useful when the number of iterations is not known beforehand.

### Program 9: Program to Display Day Name from Number (case statement)

Code:

```
#!/bin/bash
read day
echo "enter the number of day"
echo $day
case $day in
1) echo "monday" ;;
2) echo "tuesday" ;;
3) echo "wednesday" ;;
4) echo "thursday" ;;
5) echo "friday" ;;
6) echo "saturday" ;;
7) echo "sunday" ;;
*) echo "error" ;;
esac
```

Run: ./script.sh

Expected Output:

```
2
enter the number of day
2
tuesday
```

Questions & Answers:

**Q: What is the 'case' statement in shell?**

A: The case statement is a multi-way branch control structure. It matches a variable against patterns and executes the matching block.

**Q: What does '\*' mean in a case statement?**

A: The '\*' is the default case — it matches anything that didn't match the previous patterns.

**Q: What terminates each case option?**

A: Double semicolons ';' terminate each case option.

**Q: What is the closing keyword of a case statement?**

A: The case statement is closed with 'esac' (case spelled backwards).

### Program 10: Program to Find the Largest of Three Numbers

Code:

```
#!/bin/bash
echo "enter first number"
read num1
echo "enter second number"
read num2
echo "enter third number"
read num3

if [ $num1 -gt $num2 ] && [ $num1 -gt $num3 ];
then echo "largest"
elif [ $num2 -gt $num1 ] && [ $num2 -gt $num3 ];
then echo "largest"
else echo "$num3 is largest"
fi
```

Run: ./script.sh

**Expected Output:**

```
enter first number
1
enter second number
2
enter third number
5
5 is largest
```

**Questions & Answers:**

**Q: How do you use logical AND in shell?**

A: Using && between two conditions: [ condition1 ] && [ condition2 ] or [[ condition1 && condition2 ]]

**Q: What is 'elif' in shell?**

A: elif (else if) allows checking additional conditions when the initial if condition is false.

**Q: How do you find the largest of three numbers in shell?**

A: Compare each pair using -gt (greater than) with nested if-elif-else conditions.

---

**Program 11: Program to Search a Pattern in a Given File**

**Code:**

```
#!/bin/bash
echo "enter the file name"
read file
echo "enter the pattern to search"
read pattern
grep "$pattern" "$file"
```

Run: ./script.sh

**Expected Output:**

(Lines matching the pattern from the file are displayed)

**Questions & Answers:**

**Q: What is 'grep' used for?**

A: grep (Global Regular Expression Print) searches for a pattern in a file and prints lines that contain the match.

**Q: What flag makes grep case-insensitive?**

A: grep -i 'pattern' filename makes the search case-insensitive.

**Q: How do you count the number of matching lines with grep?**

A: Using grep -c 'pattern' filename

---

**■ Viva Questions – Shell Programming**

★ **What is a shell in Linux?**

→ A shell is a command-line interpreter that provides a user interface for the Unix/Linux OS. It takes commands from the user and passes them to the OS kernel for execution. Common shells: bash, sh, zsh, fish.

★ **What is the difference between sh and bash?**

→ sh (Bourne Shell) is the original Unix shell with basic features. bash (Bourne Again SHell) is an enhanced version with more features like arrays, advanced string manipulation, and arithmetic expansion.

★ **What is the difference between single quotes and double quotes in shell?**

→ Double quotes allow variable expansion (\$var is replaced by its value). Single quotes treat everything literally — no variable substitution.

★ **What is a process in Linux?**

→ A process is an instance of a running program. Each process has a unique PID (Process ID).

★ **What is the 'chmod' command?**

→ chmod (change mode) changes the file permissions. chmod +x makes a file executable. chmod 755 gives rwxr-xr-x permissions.

★ **What is stdin, stdout, and stderr?**

→ stdin (0) is standard input, stdout (1) is standard output, stderr (2) is standard error. These are the three default data streams in Linux.

★ **What is piping in shell?**

→ Piping (|) connects the output of one command as the input to another. Example: ls | grep '.txt' lists files and filters those with .txt

★ **What is the difference between > and >> in shell?**

→ > redirects output to a file, overwriting it. >> appends output to the end of a file without overwriting.

★ **What does 'exit 0' mean in a shell script?**

→ exit 0 means the script terminated successfully. Any non-zero exit code (exit 1, exit 2, etc.) indicates an error.

★ **What is the use of 'sed' in shell?**

→ sed (Stream Editor) is used for text substitution, deletion, and other transformations on files or streams.  
Example: sed 's/old/new/g' file.txt

## EXPERIMENT 5: FAMILIARISATION OF BASIC LINUX COMMANDS

**Problem Definition:** To understand and execute basic Linux commands.

### ■ Components Required

1. A computer or virtual machine with a Linux-based OS
2. Terminal access
3. Internet connection

### ■ Theory

Linux commands are the primary way to interact with the Operating System via the terminal. Each command serves a specific purpose, such as managing files, monitoring the system, or configuring user permissions.

| #  | Command | Purpose                                                      | Usage/Example                                                 |
|----|---------|--------------------------------------------------------------|---------------------------------------------------------------|
| 1  | ls      | List files and directories in the current directory          | ls<br>ls -a (show hidden)<br>ls -l (long format)              |
| 2  | ls -a   | Show hidden files (starting with .)                          | ls -a                                                         |
| 3  | mkdir   | Create a new directory                                       | mkdir myFolder                                                |
| 4  | cp      | Copy files or directories                                    | cp file1.txt file2.txt<br>cp file.txt /path/to/dest/          |
| 5  | mv      | Move or rename files/directories                             | mv file.txt newname.txt<br>mv file.txt /path/to/destination/  |
| 6  | grep    | Search for a specific string in files                        | grep 'pattern' filename.txt<br>grep -i 'pattern' filename.txt |
| 7  | rmdir   | Delete an empty directory                                    | rmdir myFolder                                                |
| 8  | chmod   | Change file or directory permissions                         | chmod +x script.sh<br>chmod 755 file.txt                      |
| 9  | passwd  | Change a user's password                                     | passwd<br>passwd username                                     |
| 10 | history | Display the list of previously entered commands              | history<br>history   grep 'command'                           |
| 11 | dmesg   | Display kernel messages / boot messages                      | dmesg<br>dmesg   tail (view recent messages)                  |
| 12 | cpuinfo | Display CPU information                                      | cat /proc/cpuinfo                                             |
| 13 | uname   | Display system/kernel information                            | uname -a<br>uname -r (kernel release)                         |
| 14 | du      | Display disk and storage management; display directory sizes | du<br>du -h (human readable)                                  |
| 15 | fdisk   | Used to manage disk partitions (human/advanced format)       | fdisk /dev/sda                                                |
| 16 | time    | Measure the execution time of a command                      | time bash script.sh<br>time copy code<br>time ls              |
| 17 | write   | Sends a message to another user on the same system           | write username                                                |

## ■ Expected Questions & Answers

### Q: What is the command to list all files including hidden ones?

A: `ls -a` — The `-a` flag shows all files, including hidden files that begin with a dot (`.`)

### Q: What is the difference between 'cp' and 'mv'?

A: 'cp' copies a file (original remains). 'mv' moves a file (original is removed from source) or renames it.

### Q: How do you create a new directory?

A: Using the 'mkdir' command followed by the directory name. Example: `mkdir projects`

### Q: What does 'chmod 755' mean?

A: 7 (rwx for owner), 5 (r-x for group), 5 (r-x for others). Owner can read, write, execute; group and others can only read and execute.

### Q: What does 'grep' do?

A: `grep` (Global Regular Expression Print) searches for a text pattern within files and displays matching lines.

### Q: What does 'uname -a' display?

A: All system information: kernel name, hostname, kernel release, kernel version, machine hardware name, and operating system.

### Q: What is 'dmesg' used for?

A: `dmesg` displays the kernel ring buffer messages, which include boot messages and hardware initialization information.

### Q: What is the difference between 'rmdir' and 'rm -r'?

A: 'rmdir' removes only empty directories. 'rm -r' removes a directory and all its contents recursively.

### Q: What does 'du -sh' display?

A: 'du -sh foldername' displays the total disk usage of a directory in human-readable format (KB, MB, GB).

### Q: What does the 'history' command do?

A: It displays a numbered list of previously entered commands, allowing you to review, repeat, or search past commands.

### Q: What is the 'fdisk' command used for?

A: `fdisk` is used to manage disk partitions — creating, deleting, and modifying partitions on a disk.

### Q: What does 'write username' do?

A: It allows you to send a real-time message to another logged-in user on the same Linux system.

### Q: What is the 'time' command?

A: The 'time' command measures the time taken to execute a command or script, showing real time, user CPU time, and system CPU time.

### Q: How do you view CPU information in Linux?

A: Using 'cat /proc/cpuinfo' which shows detailed processor information.

## ■ Viva Questions – Linux Commands

### ★ What is the difference between absolute path and relative path?

→ Absolute path starts from the root directory (`/`). Example: `/home/user/file.txt`. Relative path is relative to the current directory. Example: `./file.txt` or `../folder/file.txt`

### ★ What does 'sudo' mean?

→ `sudo` (Superuser Do) allows a permitted user to run commands as the superuser (root). It's used for system administration tasks.

### ★ What is the root directory in Linux?

→ The root directory `/` is the top-level directory in the Linux file system hierarchy. All other directories and files stem from it.

★ **What is the difference between 'ls -l' and 'ls -a'?**

→ 'ls -l' shows files in long format (permissions, owner, size, date). 'ls -a' shows all files including hidden ones.

★ **How do you search for a file in Linux?**

→ Using 'find' command: `find /path -name 'filename'`. Also `locate filename` if the locate database is updated.

★ **What is a pipe (|) in Linux?**

→ A pipe redirects the output of one command as the input to another. Example: `ls -l | grep 'txt'` lists all .txt files.

★ **What does 'cat' command do?**

→ cat (concatenate) displays the content of a file, creates files, or concatenates multiple files. Example: `cat file.txt`

★ **What are file permissions in Linux?**

→ Linux file permissions are represented as rwx (read, write, execute) for three categories: owner, group, and others. They control who can access a file and how.

★ **What is the difference between a file and a directory in Linux?**

→ A file stores data (text, binary, etc.). A directory is a container that holds files and other directories. In Linux, everything is treated as a file.

★ **What is the purpose of 'chmod +x'?**

→ `chmod +x` adds execute permission to a file for all users (owner, group, others), making it executable as a program or script.

**Result:** Successfully executed the listed Linux commands.

---

— End of Lab Record —